

OPSsig Self-Evaluation Report (OPSsig-256, Performance-Optimized Implementation)

1 Basic Algorithm Information

Item	Value
Algorithm category	Digital Signature
Algorithm name	OPSsig
Sub-function	Key generation, signature generation, and signature verification
Implementation version	Performance-Optimized Implementation
Submission version label	x86 Performance-Optimized Implementation
Parameter set / security level	OPSsig-256

2 Evaluation Environment

Item	Value
Processor model	Intel(R) Xeon(R) Platinum 8575C
Processor frequency (reported)	2800.000 MHz
Logical cores	4
Installed memory	15698744 KiB (14.97 GiB)
Architecture	x86_64
Operating system	Ubuntu 22.04.5 LTS
Kernel version	5.15.0-71-generic
Compiler version	gcc (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
Build tool version	GNU Make 4.3; cmake version 3.22.1
GNU time version	time (GNU Time) UNKNOWN
AVX2 support reported by host	yes
Evaluation start time (UTC)	2026-06-29T13:55:22Z
Evaluation end time (UTC)	2026-06-29T13:56:08Z

3 Functional Tests

The functional evaluation was executed through the current self-evaluation entry point `make selfeval`. The following checks passed:

- Known-answer test (KAT) generation completed successfully and wrote vectors to:
`Test_Vectors/avx2/OPSsig-256/KAT_SIG_OPSsig-256-avx2.txt`
- Signature correctness regression test completed successfully according to the captured log.
- The correctness log reports the exported API sizes shown below and ends in a pass status.

Correctness log excerpt

```
INFO: CRYPTO_PUBLICKEYBYTES = 3392
INFO: CRYPTO_SECRETKEYBYTES = 5056
INFO: CRYPTO_BYTES = 5540
OK: sign/verify test passed (10000 iterations)
```

KAT generation log excerpt

```
Files have been saved in ../../../../Test_Vectors/avx2/OPSsig-256.
```

4 Performance Tests

Performance measurements were collected for a fixed 64-byte message, in line with the current x86 self-evaluation guideline for digital signatures. Each core operation was sampled 1000 times. The table below reports average latency, average cycle count, and throughput.

Operation	Samples	Avg. ns	Avg. cycles	Ops/s
Key generation	1000	428115.97	1198417.06	2335.82
Sign (64 B)	1000	680858.34	1906026.51	1468.73
Verify (64 B)	1000	430990.19	1206488.66	2320.24

Supplemental Micro-Benchmarks

Operation	Avg. ns	Avg. cycles	Ops/s
polyvec_matrix_expand	266374.61	745581.20	3754.11
poly_uniform_eta	26634.67	74407.86	37545.05
poly_uniform_gamma1	56779.58	158809.04	17611.97
poly_ntt	4694.04	12994.68	213035.92
poly_invntt_tomont	4355.99	12042.31	229568.94
poly_pointwise_montgomery	174.69	345.56	5724589.98
poly_challenge	3880.38	10713.87	257706.45

5 Resource Consumption Tests

Static Memory Footprint

Static memory usage was estimated from the loaded binary sections of `test_static_mem_OPSSig` using the `size` tool. This load-only binary links the full implementation but intentionally executes no key generation, signing, or verification logic. The section totals capture code, read-only constants, initialized global data, and uninitialized global data, which corresponds to the guideline item “basic memory usage after loading but before executing algorithm operations”.

Metric	Value (Bytes)	Value (MiB)
Text segment (.text)	55994	0.05
Read-only data (.rodata)	5988	0.01
Initialized data (.data)	16	0.00
Uninitialized data (.bss)	232	0.00
Total static footprint	62230	0.06

Peak Runtime Memory

Runtime memory was measured over 100 iterations of key generation, signature generation, and verification using a fixed 64-byte message.

Metric	Value (Bytes)	Value (MiB)
Peak RSS from <code>getrusage</code>	2260992	2.16
VmPeak from <code>/proc/self/status</code>	2924544	2.79
VmHWM from <code>/proc/self/status</code>	1048576	1.00

6 Transmission and Storage Overheads

Artifact	Size (Bytes)	Source
Public key	3392	Exported API size
Secret key	5056	Exported API size
Signature buffer bound	5540	CRYPTO_BYTES
Signature test message	64	Fixed performance/memory input

7 Implementation Notes

Item	Value
Third-party dependencies	No external third-party cryptographic library is linked by the evaluated build. The evaluation uses the system compiler and standard build utilities already recorded above.
Randomness generation	Deterministic DRNG support from <code>drng.c</code> is used by the self-evaluation drivers; fixed seeds are used where reproducibility is required.
Compilation flags	<code>-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra</code>
Target ISA / portability note	x86-64 with AVX2 enabled; this version uses AVX2 intrinsics and the dedicated assembly source file <code>pointwise.S</code> .
Optimization approach	Aggressive optimization using <code>-O3</code> , AVX2 vectorization, x86-64 tuning, LTO, and frame-pointer omission. POPCNT is not enabled in the default evaluated build.

8 Raw Evidence Index

Evidence item	Location / command
Implementation directory	Implementations/Optimized_Implementation/ OPSsig-256-avx2
Primary evaluation command	make -C Implementations/Optimized_ Implementation/OPSsig-256-avx2 selfeval
Self-evaluation orchestrator	Self_Evaluation/scripts/run_self_eval.sh
Environment collection script	Self_Evaluation/scripts/collect_env.sh
Static memory script	Self_Evaluation/scripts/measure_mem.sh
Metadata CSV	Self_Evaluation/results/avx2/OPSsig-256/ latest/metadata.csv
Environment CSV	Self_Evaluation/results/avx2/OPSsig-256/ latest/environment.csv
Correctness log	Self_Evaluation/results/avx2/OPSsig-256/ latest/correctness.log
Performance CSV	Self_Evaluation/results/avx2/OPSsig-256/ latest/speed.csv
Static memory CSV	Self_Evaluation/results/avx2/OPSsig-256/ latest/memory_static.csv
Runtime memory CSV	Self_Evaluation/results/avx2/OPSsig-256/ latest/memory_runtime.csv
KAT log	Self_Evaluation/results/avx2/OPSsig-256/ latest/kat.log
Generated KAT vectors	Test_Vectors/avx2/OPSsig-256/KAT_SIG OPSsig- 256-avx2.txt
Static-memory binary	Implementations/Optimized_Implementation/ OPSsig-256-avx2/test_static_mem OPSsig
Input data note	Fixed 64-byte message for performance and memory tests; reproducible seeded DRNG for correctness and KAT generation.